AUTOMATED README GENERATION FROM A SINGLE SCRIPT

By Cole Lavigne, Hiral Ankur Choksi, and Varun Raghunath Reddy

CSE 5095: AI for Software Engineering

PROBLEM

- Developers love writing code, not READMEs.

- Small repositories can lack usable documentation.

- Manual documentation = repetitive, inconsistent, error-prone.

- Core question: *Can AI understand code well enough to document it automatically?*



MOTIVATION (MIDTERM RECAP)

- **Why it matters:** Reproducibility • Reusability • Collaboration

Research Questions:

Can LLMs summarize *entire scripts*, not just snippets?

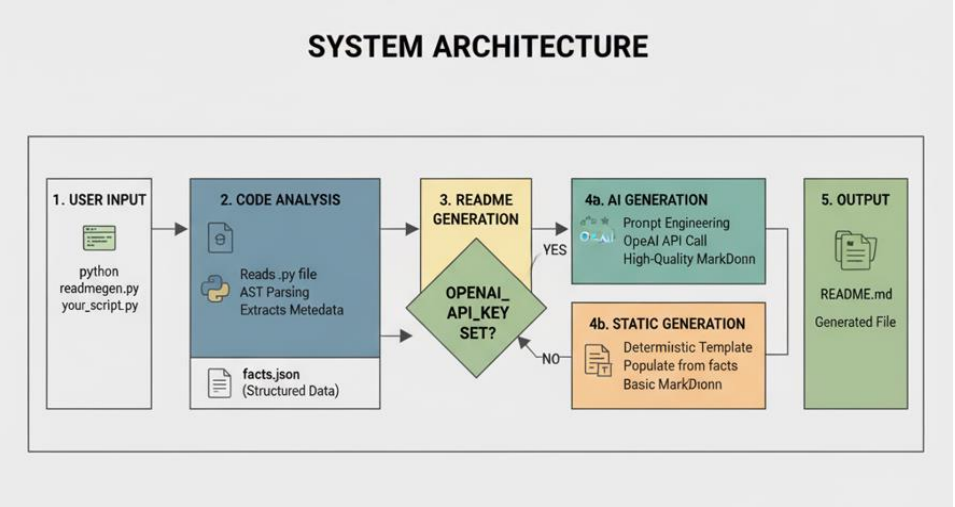
Does static analysis reduce hallucinations?

What hybrid design best balances structure and fluency?

Expected Outcome: `readmegen` – a hybrid AI +static analysis tool for auto-generating README.mdfiles.

MIDTERM CHECKPOINT STATUS

- Built static analyzer (imports, CLI flags, docstrings)
- Prompt engineering implemented (with JSON-structured facts)
- AI generation using GPT-4o-mini integrated
  - Was still newly implemented and had bugs
- Basic static fallback created
- Evaluation and LLM fine-tuning/prompt engineering was not yet complete
  - This became the core of our work this second half of the semester



System architecture of midterm project

INITIAL PLAN (RECAP)

- Build a hybrid README generator:
  - Static analysis → extract structured facts
  - LLM → write polished documentation
  - Static fallback → guaranteed output if LLM failed
- Compare LLM outputs to human reference READMEs
- Explore grounded prompt engineering to reduce hallucinations

FINAL SYSTEM ARCHITECTURE

- Multi-provider generation
  - OpenAI/Gemini (with other models currently being worked on integrating)
- Unified judge (always using OpenAI)
- Strict fact-only grounding + improved LLM prompt engineering
- Automatic error handling for scripts, model failures, missing flags, etc
- Automated evaluation script
  - Run any LLM through all sample scripts (20 total)
  - Generates READMEs for each script
  - Evaluates clarity, completeness, helpfulness, dependency precision/recall, CLI flag detection

IMPLEMENTATION UPGRADES SINCE MIDTERM

- Midterm:
  - Inputs:
    - Python script
    - Extracted facts from python script we created (analyze.py) into JSON form
  - Throws inputs into LLM prompt engineered script
  - Output:
    - Generated README.md file
  - No evaluation was done at this time (visual results only)

- Final:
  - Significantly improved LLM's ReadME output + got rid of fallback method
  - Added evaluation frame scoring 3 dimensions via OpenAI:
    - Clarity
    - Completeness
    - Helpfulness
  - Additional additions
    - Dependency precision/recall
    - CLI completeness score
    - Support for additional API keys (OpenAI, Gemini, soon Groq, looking into a few more)
    - Allows for cross-model comparisons

NEW DESIGN ADAPTATIONS

- Removed heuristic fallback for evaluation
  - LLM-only judge and ReadME generation
- Designed improved extraction rules for CLI flags, aliases, and defaults

- Fact-grounded prompts aim to reduce unsupported claims; human review remains necessary.

- Added error reporting to catch any missing flags or parsing failures
  - Through our automated evaluation workflow
- LLMs fully tested through the workflow at this time:
  - Gemini and OpenAI
  - Looking to expand to further LLMs before the end of the semester

FINAL ARCHITECTURE SUMMARY

01 Analyze the Python script

Extract imports, CLI flags, docstrings, and project metadata.

02 Build the grounded prompt

Supply structured facts through a provider adapter.

03 Generate Markdown

Request documentation from the selected language model.

04 Evaluate and inspect

Compare dependencies, CLI coverage, and judged document quality.

EVALUATION PLAN + OVERVIEW

- Our evaluation includes the following:
  - 20 paired Python scripts + high-quality ground-truth READMEs
  - Each README is scored by OpenAI judge for:
    - Clarity (1-5)
    - Completeness (1-5)
    - Helpfulness (1-5)
  - Factual consistency also checked for:
    - Dependency precision/recall
    - CLI flag detection
  - This is all automated and reports via JSON and a summary markdown

FINAL EVALUATION RESULTS (OPENAI JUDGE, OPENAI GENERATOR)

Recorded metric	Value
Dependency precision	1.000
Dependency recall	0.875
CLI completeness	0.900
Clarity	3.850 / 5
Completeness	3.200 / 5
Helpfulness	3.250 / 5

Saved academic evaluation

20 evaluated samples; 1 skipped due to an error.

Generation and judge: gpt-4o-mini.

Strong dependency precision and CLI coverage; explanation completeness remains limited.

Historical results, not a fresh benchmark.

EVALUATION LIMITATIONS

- Small sample: 20 evaluated scripts and one recorded error.

- The same model generated and judged documents; independent human assessment is needed.

- Historical AST and no-AST runners use different metric rules. Their scores do not establish a controlled causal comparison.

- Models, SDKs, references, and extracted facts can change or be imperfect.

HIGH PERFORMING SCRIPT – SEAM_CARVER.PY

Seam Carver

Image resizing via intelligent seam carving.

License MIT python 3.7+

Overview

This tool implements seam carving, an algorithm for content-aware image resizing. Given an input image, the script identifies and removes low-energy "seams" (paths of least importance) to reduce the image size along a specified axis.

Notable features:

- Resizes images by removing seams along the x or y axis.
- Supports saving intermediate images at specified intervals.
- Can highlight the removed seam on intermediate images for visualization.
- Optionally pads the resized image to match the original dimensions.

Requirements

- Python 3.7+
- Any OS supporting the required Python packages

Dependencies

Python Standard Library

- `__future__`
- `argparse`
- `os`
- `sys`

Third-Party Packages

- `PIL`
- `numba`
- `numpy`
- `tqdm`
- `groq`
- `openai`
- `energy_functions`
- `utils`

Installation

Install the necessary dependencies using `pip`:

```
pip install -r ..\requirements.txt
```

Alternatively, for an editable install:

```
pip install -e .
```

Usage

```
python seam_carver.py input_file <value> --axis <value> --pixels <value> --output <value> --interval <value> --border <value> -
```

Flags

Flag	Alias	Type	Required	Default	Help
`input_file`		`str`			
`--axis`	`-a`	`str`	True		What axis to shrink the image on. (`x` , `y`)
`--pixels`	`-p`	`int`	True		How many pixels to shrink the image by.
`--output`	`-o`	`str`			What to name the new cropped image.
`--interval`	`-i`	`int`			Save every i intermediate images.
`--border`	`-b`	`bool`			Whether or not to pad the cropped images to the size of the original
`--show_seam`	`-s`	`bool`			Whether to highlight the removed seam on the intermediate images.

Examples

1. Resize `image.jpg` by removing 50 pixels along the x-axis, saving the output as `resized.jpg`:

```
python seam_carver.py input_file image.jpg --axis x --pixels 50 --output resized.jpg
```

2. Resize `image.png` by removing 25 pixels along the y-axis, saving intermediate images every 5 iterations with highlighted seams, and padding the final image:

```
python seam_carver.py input_file image.png --axis y --pixels 25 --output resized.png --interval 5 --show_seam --border
```

Saved evaluation

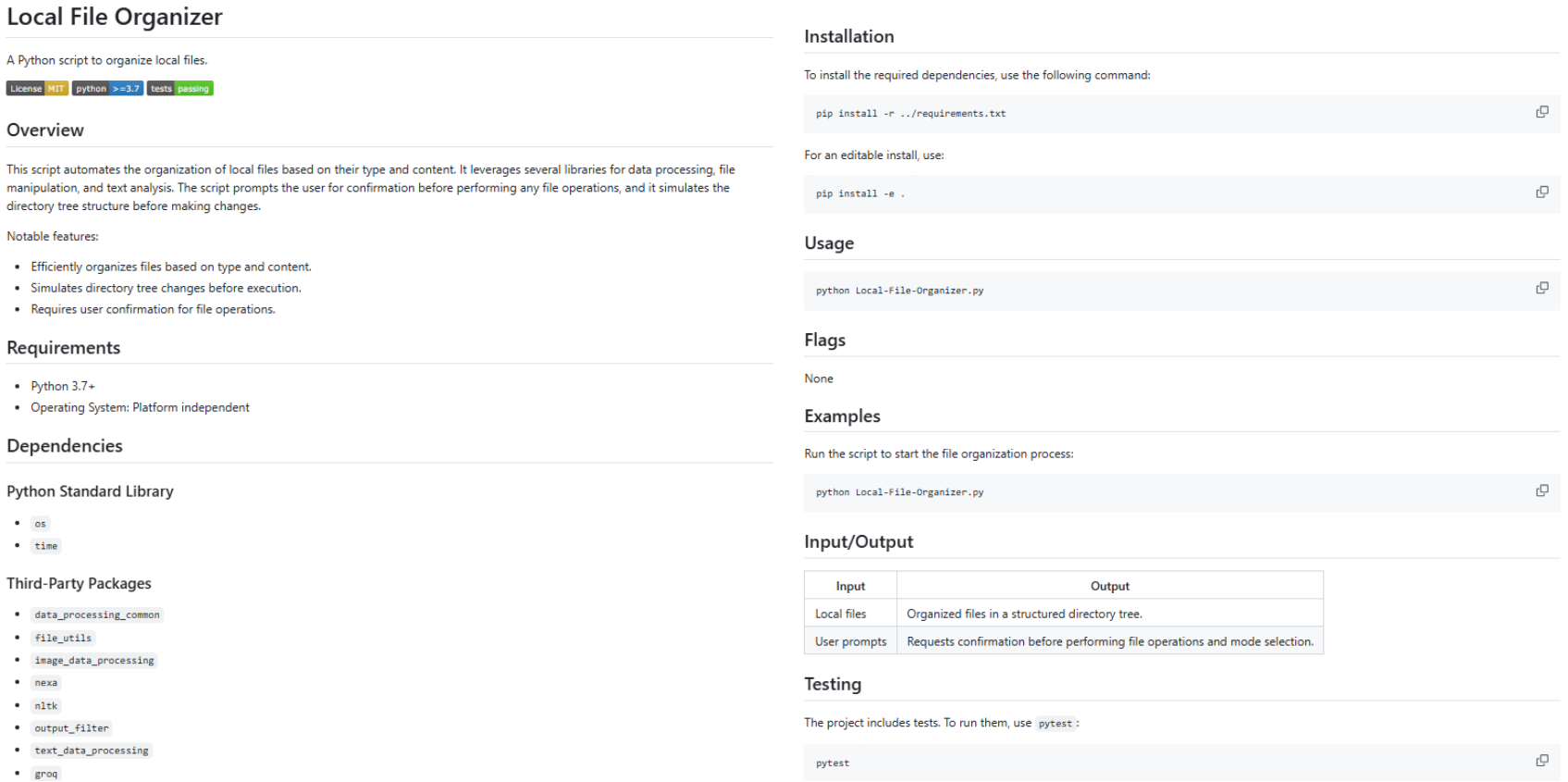
CLI flags: 12 / 12

Clarity: 4 / 5
Completeness: 3 / 5
Helpfulness: 4 / 5

Full flag coverage does not guarantee complete documentation.

Original output excerpts shown at left.

LOW-PERFORMING SCRIPT – LOCAL_FILE_ORGANIZER.PY



Local File Organizer

A Python script to organize local files.

License MIT python >=3.7 tests passing

Overview

This script automates the organization of local files based on their type and content. It leverages several libraries for data processing, file manipulation, and text analysis. The script prompts the user for confirmation before performing any file operations, and it simulates the directory tree structure before making changes.

Notable features:

- Efficiently organizes files based on type and content.
- Simulates directory tree changes before execution.
- Requires user confirmation for file operations.

Requirements

- Python 3.7+
- Operating System: Platform independent

Dependencies

Python Standard Library

- `os`
- `time`

Third-Party Packages

- `data_processing_common`
- `file_utils`
- `image_data_processing`
- `nexa`
- `nltk`
- `output_filter`
- `text_data_processing`
- `groq`

Installation

To install the required dependencies, use the following command:

```
pip install -r ../requirements.txt
```

For an editable install, use:

```
pip install -e .
```

Usage

```
python Local-File-Organizer.py
```

Flags

None

Examples

Run the script to start the file organization process:

```
python Local-File-Organizer.py
```

Input/Output

Input	Output
Local files	Organized files in a structured directory tree.
User prompts	Requests confirmation before performing file operations and mode selection.

Testing

The project includes tests. To run them, use `pytest`:

```
pytest
```

Saved evaluation

Dependency recall: 0.28
CLI completeness: 1.00

Clarity: 3 / 5
Completeness: 3 / 5
Helpfulness: 3 / 5

Static facts can miss dynamic behavior and broader application context.

Original output excerpts at left.

KEY EVALUATION TAKEAWAYS

- Dependency precision: 1.000; recall: 0.875 in the saved run.

- CLI completeness: 0.900 overall, with variation across individual scripts.

- Clarity was rated higher than completeness and helpfulness.

- Grounding provides traceable input facts; these records alone do not prove hallucination reduction.

- Generated README files still need human review.

DISCUSSION

- Structured code facts provide a practical foundation for script-level documentation.

- Usage examples, configuration, and runtime behavior can require context beyond one source file.

- A grounded prompt cannot guarantee factual accuracy or complete coverage.

- Controlled comparisons require aligned metrics, matched inputs, and independent review.

CHALLENGES + LIMITATIONS

- Challenges:
  - Complex CLI parsers lead to inconsistent extraction
    - Has improved drastically since midterm
  - Distinguishing between optional/required flags
  - Obtaining high completeness/helpfulness from scripts with limited docstrings/comments
- Limitations:
  - Only tested on single script Python scripts
  - Dataset size: 20 scripts
    - Would like to expand to 50+ scripts for deeper empirical study
  - Ground truth READMEs are generally consistent, but have occassional bad results due to evaluation noise

REMAINING WORK

- Align AST and no-AST measurement rules before comparing quality.

- Run fresh experiments with versioned environments and independent human review.

- Broaden the sample collection and document sample provenance.

- Evaluate the influence of imports, docstrings, and CLI metadata.

- Explore multi-file context and richer runtime documentation.

These are proposed improvements, not completed experiments.

CONCLUSION

- Built end-to-end pipeline for automated README generation that includes:
  - Static analysis
  - Fact grounding
  - AI generation of READMEs based off of our prompt engineering efforts
    - Multiple LLMs available
  - Automated scoring and empirical analysis
- This project bridges research and is a practical tool for README generation for single Python scripts